

AI Weatherwise API

Student Project Report

Project Team: Mastani I · Priya Bharathi T · Sharmila V · Uzma Farheen J · Vishali M

PROJECT OVERVIEW

Software Engineering & Applied AI Practicum

ABOUT THE PROJECT

The **AI Weatherwise API** turns weather data into clear summaries and practical outdoor advice. It uses Node.js and Express to handle requests, Google Gemini to create guidance, MongoDB to store records, and Postman to test the service.

Backend: Node.js · AI: Google Gemini · Database: MongoDB · Testing: Postman

Abstract

Weather apps often show numbers such as temperature, humidity, and pressure without explaining what they mean for daily plans. The **AI Weatherwise API** accepts weather details and turns them into useful summaries and suggestions with the help of an AI model.

The project uses **Node.js** and **Express** to handle API requests, **Google Gemini** to create natural-language advice, and **MongoDB** to store request records. The team built and organized the endpoints, checked the data, and tested the service with **Postman**.

In the tested example, a request for 18°C and clear skies received an HTTP 200 OK response with outdoor and clothing suggestions. This first version provides a starting point for future mobile apps and other services that can offer weather advice in context.

Keywords: Weather API; AI advice; Node.js; Google Gemini; MongoDB; Postman testing.

Contents

Project overview and objectives

Team responsibilities

Software and technology stack

System workflow and API integration

Development and implementation process

Error handling and reliability

Testing approach and evidence

Current results and limitations

Future outcomes and roadmap

Conclusion

Project Overview and Objectives

Weather services often provide numbers that are hard to turn into everyday decisions. The AI Weatherwise API aims to make weather information easier to use by accepting weather details and returning clear summaries and practical suggestions.

The project focuses on five goals:

- 1. Accept weather data:** Receive details such as temperature, chance of rain, sky conditions, and wind through JSON requests.

- 2. Check requests:** Use Node.js to check that incoming data has the expected format and values.

- 3. Create useful advice:** Use Google Gemini to turn weather details into clear summaries and suggestions for activities, travel, or clothing.

- 4. Keep records:** Use MongoDB to store request details and service logs.

- 5. Test the API:** Use Postman to check responses, data formats, and how the service handles invalid requests.

Project goal: Build a separate service that other applications can use without exposing private prompts or credentials.

Team Responsibilities

The team shared the work across project leadership, API integration, development, reliability, testing, documentation, and presentation.

Mastani I · Team leader and API integration

Priya Bharathi T · Backend development

Sharmila V · Error handling

Uzma Farheen J · Documentation and testing

Vishali M · Presentation development

Software and Technology Stack

The project uses a small set of tools to build, run, store data for, and test the API.

Table 1. Tools used in the project and their roles.

Technology	Role in the Project
Node.js	Runs the server and handles API requests.
Gemini	Creates weather summaries and suggestions using information from the request.
MongoDB	Stores weather requests and related records.
Postman	Sends test requests and helps check responses and data formats.
Code Editor	Used to write, organize, and debug the project code.

How the tools work together: The API receives and handles requests with Node.js, sends weather information to Gemini for advice, and can store request records in MongoDB.

System Workflow and API Integration

The service handles a request in a few steps:

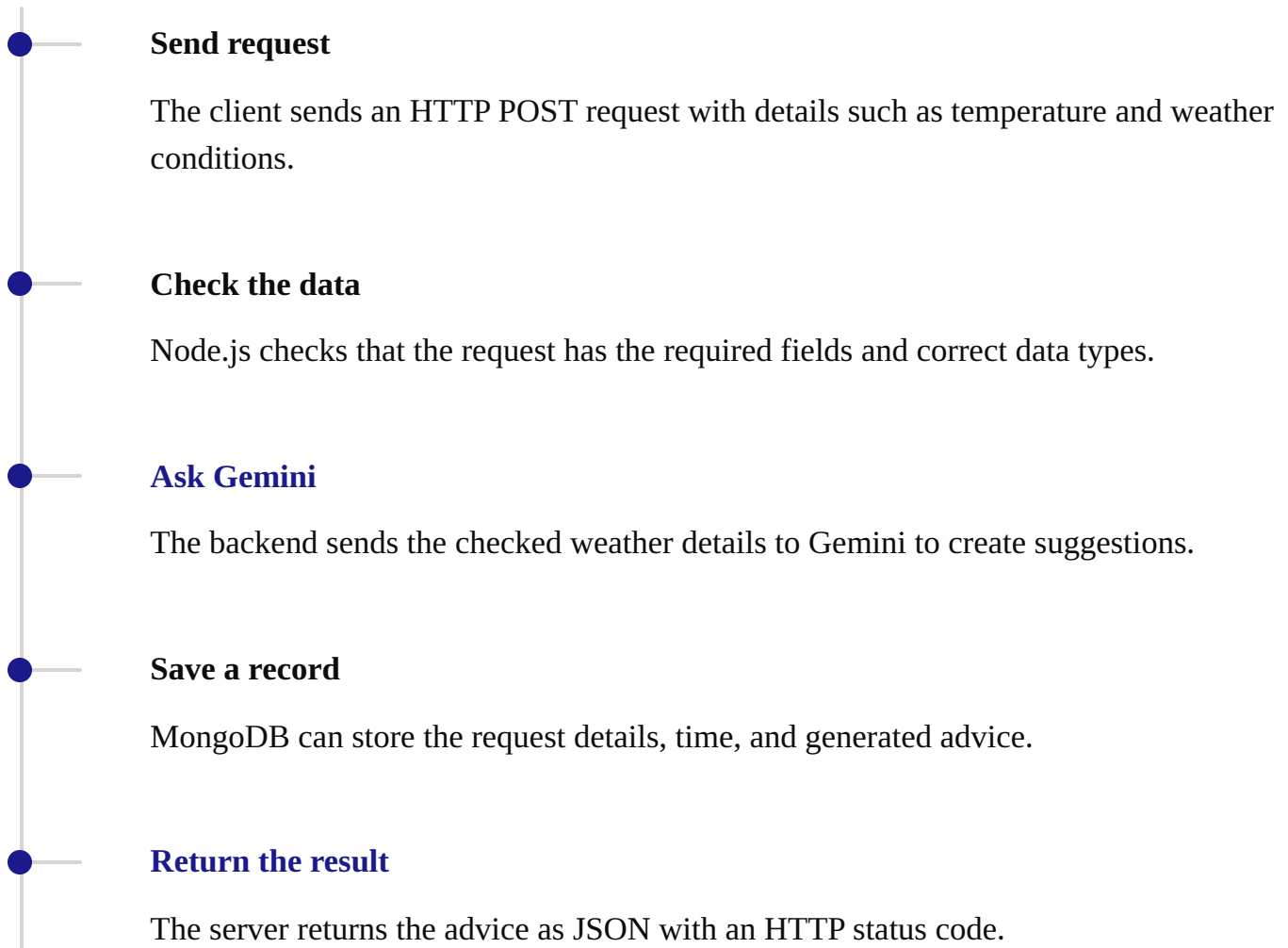


Figure 1. Steps in a weather advice request.

Request and response: A request includes a numeric **temperature** in Celsius and a **conditions** value such as “Clear” or “Rainy.” The response includes generated advice and status information.

Development and Implementation Process

The team built the project in stages, improving and testing it as work progressed.

01 Project setup

Set up the project

Created the Node.js project and added tools such as Express, the Google Gen AI SDK, MongoDB libraries, and dotenv.

02 Code organization

Organize the code

Separated routes, controller logic, AI prompts, and database connection code into modules.

03 API improvement

Improve the API

Updated the recommendation logic, worked on consistent replies, and protected private settings with environment variables.

04 Testing

Run and test the service

Ran the server locally and used Postman to check requests, responses, and timing.

05 Review and completion

Review and finish

Reviewed the generated advice, improved error handling, and completed the project documentation.

Error Handling and Reliability

The API needs to handle missing or incorrect data, network problems, and issues with the AI service without crashing or showing confusing errors.

Area	How it is handled
Input checks	The API checks for required fields such as temperature and conditions. Missing, invalid, or out-of-range values receive a 400 Bad Request response.
HTTP status codes	The API uses 200 OK for success, 400 Bad Request for invalid requests, 500 Internal Server Error for unexpected problems, and 503 Service Unavailable when the AI service cannot respond.
AI service issues	Calls to Gemini use error handling and time limits. If the service is unavailable or reaches a limit, the API records the issue and can return backup advice without exposing technical details.
Logs	Errors and recovery events can be recorded to help the team understand problems and improve the service.

Keeping credentials private: API keys and database connection strings are stored in environment variables, not sent to users in responses.

Testing Approach and Evidence

The team used **Postman** to test the API, check response times, and review the advice it returned. Tests checked that the service reads JSON requests, contacts Gemini, and returns structured recommendations with appropriate status codes.

TEST EXAMPLE

Target Endpoint	POST /weather-recommendation
HTTP Status	200 OK
Test Conditions	Temperature: 18°C · Sky: Clear
Input Format	Structured JSON Body
Output Behavior	Outdoor and clothing suggestions

Result: In the test shown, a POST request with a temperature of 18°C and clear skies returned a 200 OK status and structured suggestions. This checked that the Node.js service and Gemini worked together for this example.

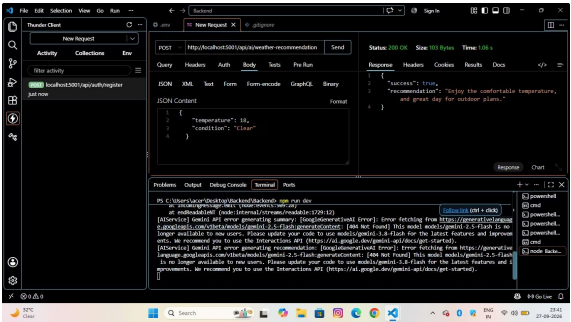


Figure 2. Postman test showing a 200 OK response.

Testing summary: The test shows that the service accepted the example request and returned advice in the expected format.

Future of AI Weatherwise

AI Weatherwise can grow from a useful starting point into a smarter everyday guide. With a mobile app, richer forecasts, and advice shaped around each person's location and plans, it could make weather easier to act on.

Next, the team can expand testing across changing conditions and unusual inputs, then refine fallback advice and response reliability. Connecting the API to mobile, smart-home, and travel tools could bring relevant guidance into everyday plans.

With careful evaluation, privacy-conscious data handling, and feedback from users, AI Weatherwise can grow into a dependable service that helps people make informed choices as the weather changes.